TÀI LIỆU KIẾN TRÚC HỆ THỐNG VÀ HƯỚNG DẪN KỸ THUẬT

MÔN HỌC: MẠNG MÁY TÍNH (CSC10008) - HCMUS
ĐỀ TÀI: ỨNG DỤNG TRÒ CHUYỆN VÀ CHIA SẺ TỆP TIN P2P TRONG MẠNG CỤC BỘ (LAN)

---

1. TỔNG QUAN DỰ ÁN VÀ MỤC TIÊU KỸ THUẬT

Dự án xây dựng một hệ thống truyền thông thời gian thực trong mạng nội bộ (Local Area Network - LAN) theo mô hình mạng ngang hàng thuần túy (Pure Peer-to-Peer), không phụ thuộc vào bất kỳ máy chủ trung tâm (Centralized Server) nào. Tất cả các nút (máy tính) trong mạng đều đóng vai trò bình đẳng: vừa là bên yêu cầu dịch vụ (Client) vừa là bên cung cấp dịch vụ (Server).

Các mục tiêu kỹ thuật trọng tâm:
- Tự động phát hiện (Peer Discovery): Sử dụng cơ chế UDP Broadcast định kỳ 3 giây tới địa chỉ 255.255.255.255 trên cổng 9999 để phát hiện các thiết bị cùng tham gia mạng LAN mà không cần cấu hình IP tĩnh thủ công.
- Quản lý trạng thái hiện diện (Presence Management): Hỗ trợ cập nhật trạng thái Online / Away và cơ chế Heartbeat với ngưỡng Timeout 10 giây (tự động xóa peer khỏi danh bạ nếu không nhận được tín hiệu sau 10 giây).
- Truyền tin cậy qua TCP: Mỗi nút mở cổng TCP (mặc định 5000, tự động chọn cổng kế tiếp nếu bị bận) để đón nhận các kết nối nhắn tin và truyền file trực tiếp.
- Đặc tả phân khung dữ liệu (Message Framing): Giải quyết triệt để vấn đề dính gói (Packet Sticking) và cắt gói (Packet Fragmentation) trong luồng byte stream TCP bằng kỹ thuật tiền tố độ dài (Length-prefixed Framing).
- Truyền file theo khối (Chunked File Transfer): Đọc và truyền tệp tin nhị phân theo từng khối 64 KB, cập nhật tiến độ phần trăm (%) theo thời gian thực và tự động lưu vào thư mục tải về.
- Lập trình Native Socket: Sử dụng trực tiếp API Windows Sockets 2 (ws2_32) theo đúng yêu cầu học thuật của môn Mạng máy tính.

2. KIẾN TRÚC TỔNG THỂ HỆ THỐNG

Hệ thống kết hợp tối ưu giữa hai giao thức tầng giao vận (Transport Layer):
1. UDP (User Datagram Protocol) - Kênh quảng bá hiện diện:
- Sử dụng để phát thông điệp ngắn thông báo sự hiện diện tới toàn bộ dải mạng LAN qua cổng cố định 9999. Do UDP không cần thiết lập bắt tay và không tốn tài nguyên duy trì phiên kết nối, nó là lựa chọn lý tưởng cho tác vụ discovery liên tục.
2. TCP (Transmission Control Protocol) - Kênh truyền dữ liệu tin cậy:
- Sử dụng cho toàn bộ hoạt động trao đổi tin nhắn văn bản và truyền tệp tin giữa hai người dùng cụ thể (1-1 Communication). TCP đảm bảo tính toàn vẹn dữ liệu, kiểm soát luồng và sửa lỗi tự động.

Quy trình tương tác tiêu biểu giữa hai Client A và B:
- Bước 1: Client A khởi động, mở cổng lắng nghe TCP 5001, đồng thời phát định kỳ gói tin UDP: HELLO|Alice|5001|online tới 255.255.255.255:9999.
- Bước 2: Client B nhận được gói HELLO của A, tự động ghi nhận A vào danh bạ và phản hồi bằng việc phát gói HELLO của B: HELLO|Bob|5002|online.
- Bước 3: Khi A muốn gửi tin nhắn cho B, A mở socket TCP kết nối tới địa chỉ IP của B trên cổng 5002.
- Bước 4: A gửi gói tin bắt tay xác thực: AUTH|Alice|5001|.
- Bước 5: Hai bên bắt đầu trao đổi tin nhắn (định dạng MSG|len|text) và truyền tệp tin (định dạng FILE|name|size| + 64KB chunks).

3. MÔ HÌNH ĐA LUỒNG (MULTI-THREADING MODEL)

Để đảm bảo ứng dụng không bao giờ bị đơ giao diện (UI Freeze) khi thực hiện các tác vụ mạng bị chặn (Blocking I/O), hệ thống chia tách công việc thành các luồng độc lập:
1. Luồng giao diện chính (Main / UI Thread): Điều phối vòng lặp sự kiện Qt (QApplication::exec()), tiếp nhận thao tác người dùng, hiển thị danh sách peer, khung chat và thanh tiến trình truyền file. Tuyệt đối không thực hiện socket blocking trên luồng này.
2. Luồng phát UDP Broadcast (Sender Thread): Vòng lặp ngủ 3 giây và gửi thông điệp HELLO qua socket UDP.
3. Luồng thu UDP Broadcast (Listener Thread): Luôn trực tại lệnh recvfrom() trên cổng 9999, giải mã gói tin đến và cập nhật vào danh bạ peer dưới sự bảo vệ của std::mutex.
4. Luồng kiểm tra Timeout (Heartbeat Timeout Thread): Định kỳ 1 giây quét danh sách peer; peer nào có lastSeen > 10s sẽ bị tự động loại bỏ.
5. Luồng đón nhận kết nối TCP (TCP Acceptor Thread): Chờ đợi tại hàm accept() trên cổng TCP đang mở. Khi có peer kết nối, khởi tạo một đối tượng TcpConnection riêng biệt.
6. Luồng thu dữ liệu TCP (TCP Receiver Thread): Mỗi kết nối TCP sở hữu một luồng riêng chờ tại lệnh recv(), giải mã framing tin nhắn hoặc ghi dữ liệu file nhận được xuống đĩa.
7. Luồng gửi tệp tin (File Sender Worker Thread): Khi gửi tệp, luồng riêng biệt này đọc file nhị phân theo khối 64 KB và gọi send() tuần tự, tính toán phần trăm hoàn thành mà không làm gián đoạn việc nhắn tin.

4. ĐẶC TẢ CHI TIẾT GIAO THỨC TẦNG ỨNG DỤNG

Giao thức sử dụng ký tự gạch đứng | làm ký tự phân tách trường (Field Delimiter) và mã hóa văn bản chuẩn UTF-8.

4.1. Giao thức UDP Presence Broadcast:
- Cú pháp: HELLO|<name>|<tcp_port>|<status>
- Ví dụ: HELLO|Alice|5001|online hoặc HELLO|Bob|5002|away
- Chu kỳ phát: 3 giây. Ngưỡng hết hạn (Timeout): 10 giây.

4.2. Giao thức TCP Authentication Handshake:
- Cú pháp: AUTH|<name>|<tcp_port>|
- Ví dụ: AUTH|Alice|5001|
- Mục đích: Báo cho bên nhận biết định danh của bên vừa kết nối để liên kết với danh bạ peer.

4.3. Giao thức TCP Message Framing (Chống dính gói & cắt gói):
- Cú pháp: MSG|<length>|<utf8_text>
- Ví dụ: MSG|11|Chào bạn!
- Cơ chế xử lý: Bên nhận phân tích độ dài length, sau đó lặp nhận chính xác length byte trước khi chuyển tin nhắn lên giao diện.

4.4. Giao thức TCP File Transfer:
- Cú pháp Header: FILE|<filename>|<size_in_bytes>|
- Sau header là chuỗi các byte nhị phân được chia thành các khối 64 KB (65,536 bytes).
- Bên nhận mở file tại thư mục downloads/<filename> và ghi liên tục cho đến khi đủ số byte quy định trong header.

4.5. Giao thức TCP Disconnect:
- Cú pháp: BYE|
- Dùng để thông báo ngắt phiên trò chuyện một cách lịch sự trước khi đóng socket.

5. HỆ THỐNG CÔNG CỤ VÀ CÔNG NGHỆ SỬ DỤNG

Công cụ / Công nghệ	Phiên bản	Vai trò và Lý do lựa chọn
Ngôn ngữ C++	C++17	Hỗ trợ lập trình đa luồng chuẩn (std::thread, std::mutex), quản lý bộ nhớ chặt chẽ, tối ưu hiệu năng truyền nhận mạng tốc độ cao.
Trình biên dịch GCC	16.2.0 (UCRT64)	Trình biên dịch mã nguồn mở hiện đại, tương thích chuẩn Universal C Runtime của Windows 10/11, hỗ trợ đầy đủ các hàm POSIX và Win32.
Hệ thống build CMake	3.20+	Quản lý toàn bộ cấu hình biên dịch độc lập môi trường, tự động phát hiện Qt6, tự động sinh các quy tắc build song song.
Trình thực thi Ninja	1.10+	Hệ thống build tập trung vào tốc độ tối đa; đọc trực tiếp đồ thị phụ thuộc từ CMake và thực thi biên dịch đa luồng vượt trội so với Make truyền thống.
Thư viện GUI Qt6	6.12.0 Widgets	Cung cấp giao diện trực quan đa nền tảng, cơ chế Signal/Slot an toàn để chuyển dữ liệu từ luồng socket sang luồng hiển thị mà không gây xung đột.
Windows Sockets 2	ws2_32	Thư viện mạng gốc (Native Socket API) của hệ điều hành Windows, đáp ứng đúng yêu cầu học thuật của môn học (không dùng thư viện mạng bọc sẵn).
Công cụ Wireshark	Mới nhất	Dùng để bắt và phân tích gói tin mạng thực tế (UDP Broadcast cổng 9999, TCP Three-Way Handshake, TCP PSH Segments) để nghiệm thu đồ án.

6. HƯỚNG DẪN BIÊN DỊCH VÀ CHẠY ỨNG DỤNG

Thực hiện lần lượt các bước sau trong PowerShell hoặc Terminal tại thư mục gốc của dự án:

Bước 1: Khởi tạo cấu hình build với CMake và Ninja:
cmake -B build -G "Ninja" -DCMAKE_EXPORT_COMPILE_COMMANDS=ON

Bước 2: Tiến hành biên dịch toàn bộ mã nguồn:
cmake --build build

Bước 3: Chạy ứng dụng đã liên kết:
.\build\lan_chat.exe

Kiểm thử nhiều client trên cùng một máy:
Mở đồng thời 2 hoặc 3 cửa sổ terminal và cùng khởi chạy .\build\lan_chat.exe. Ứng dụng sẽ tự động chọn các cổng TCP khác nhau (5000, 5001, 5002...) và phát hiện lẫn nhau qua card Loopback.

7. HƯỚNG DẪN BẮT GÓI TIN BẰNG WIRESHARK

Để phục vụ báo cáo nghiệm thu và phân tích giao thức:

1. Khởi động Wireshark và chọn card mạng phù hợp:
   - Nếu thử nghiệm giữa các máy thực tế qua Wi-Fi / Cáp LAN: Chọn card Wi-Fi hoặc Ethernet.
   - Nếu chạy nhiều client trên cùng một máy tính: Chọn card Npcap Loopback Adapter.
2. Áp dụng bộ lọc hiển thị (Display Filter):
   udp.port == 9999 || tcp.port >= 5000
3. Quan sát các bước hoạt động:
   - Gói tin UDP Broadcast mang chuỗi HELLO|... định kỳ mỗi 3 giây.
   - Gói tin TCP bắt tay 3 bước (SYN, SYN-ACK, ACK) khi click chọn bạn chat.
   - Gói tin dữ liệu AUTH|..., MSG|... và các khối dữ liệu truyền file FILE|....

8. QUY ƯỚC QUẢN LÝ MÃ NGUỒN VÀ DÒNG LÀM VIỆC GIT

- Nhánh main là nhánh sản phẩm ổn định, luôn đảm bảo biên dịch thành công. - Mỗi tính năng được phát triển trên nhánh riêng theo quy tắc: feature/<tên-tính-năng>. - Thông điệp commit viết bằng tiếng Anh hoặc tiếng Việt chuyên nghiệp, theo chuẩn Conventional Commits (ví dụ: feat: implement udp broadcast heartbeat), tuyệt đối không chèn biểu tượng cảm xúc (emoji). - Mọi mã nguồn được tích hợp vào main thông qua hình thức Pull Request có sự đồng thuận của các thành viên.